React Conversion Plan (Strangler Pattern)

Goal: Migrate the UI to React safely and quickly, keep the existing Express backend, and defer deeper backend refactors until after the React UI stabilizes.

1) Scope and goals (concise)

- Maintain parity while improving maintainability and performance.
- Keep prod stable; deliver React in small increments.
- Do minimal backend hardening now; deeper refactor later.

2) Branching and Render deploys

- Production Web Service → branch: 9_1_25-deploy-new (prod DB)
- Preview Web Service → branch: new_mbh/REACT_Conv (staging DB)
- CORS: allow both prod and preview origins.
- Verify with /version on each service (branch + commit).

3) Minimal backend hardening now (don't block React)

- Enforce auth on protected endpoints, add simple rate limits.
- CORS allowlist (prod + preview), structured logs with request IDs.
- Basic validation (Zod/Joi) on endpoints React will hit first.
- Health/readiness endpoints; DB ping.

4) Strangler steps (React islands → full SPA)

1. Create web-react/ with Vite; output to web/react-build/.
2. Add
   to index.html; load the React bundle with the CSP nonce the server injects.
3. Add AppBridge (small pub/sub + getToken helpers) for legacy↔React interop.
4. Start with a tiny, low-risk widget; validate and ship.
5. Migrate more components/pages iteratively; feature-flag/A-B as needed.
6. Introduce React Router when most routes are React; then swap to SPA routing.
7. Optional SSR later if SEO needs it (consider Next.js then).

Note on static files: if web/ doesn't exist in REACT_Conv, either (a) move assets under web/, or (b) add a fallback in server/server.js to serve from repo root when ../web is missing.

5) Data layer and performance

- Use React Query for server state; Context for session.
- Code-split (React.lazy/Suspense); remove legacy JS as React replaces it.

6) Testing and rollout

- Unit + smoke tests for migrated pieces; Playwright for key flows.
- Contract tests for the React-used endpoints to avoid breaking changes.

- Deploy to preview first; when parity is reached, merge or flip prod to the React branch.

7) After React completes (deeper backend work)

- TypeScript + layered structure; full request/response validation; OpenAPI docs.
- Job/agent queue (Redis/BullMQ or pg-boss), metrics/OTEL traces, SLOs/alerts.
- If team/complexity warrants, consider gradual migration to NestJS behind stable API contracts.

This keeps risk low: React now + minimal safeguards, then deeper backend hardening after the UI is stable.